CSS 262: Linux Administration

Vulnerability Scanning & Patching

Lecture 13: Find It, Fix It, Verify It

📋 Today's Agenda

Part 1: The Vulnerability Lifecycle

- CVEs, CVSS scores, NVD
- Patch Tuesday, zero-days, disclosure
- Risk prioritization

Part 2: Linux Package Patching

- `apt` / `dnf` update workflows
- Unattended upgrades & dnf-automatic
- Kernel updates and reboots

Part 3: Vulnerability Scanning Tools

- OpenVAS / Greenbone
- Lynis – local hardening auditor
- Trivy for containers and filesystems
- `debsecan` / `rpm -qa` + advisories

Part 4: Patch Management at Scale

- Ansible for fleet patching
- Rollback strategies
- Compliance: CIS Benchmark checks

🔄 Quick Recap: Week 12

Docker & Containers

- Namespaces (pid, net, mnt) – process isolation
- cgroups – resource limits per container
- Dockerfile: pin versions, non-root USER, multi-stage
- `--cap-drop ALL`, `--read-only`, resource limits

Key Takeaways

- Containers share the host kernel – vulnerable kernel = vulnerable containers
- Docker socket = root backdoor
- Scan images with Trivy before deploying
- Secrets do NOT belong in Dockerfiles

🔍 **Today:** Scanning identified vulnerabilities in Week 12's containers – now we go deeper: scan the OS, prioritize findings, and patch systematically.

Part 1

🔬 The Vulnerability Lifecycle

🔬 What Is a Vulnerability?

CVE – Common Vulnerabilities and Exposures

- A **CVE ID** is a unique identifier for a publicly disclosed vulnerability
- Format: `CVE-YEAR-NUMBER` (e.g., `CVE-2021-44228` = Log4Shell)
- Maintained by MITRE, entries in the NVD
- Includes: description, affected software versions, references, CVSS score

Where CVEs Come From

- Security researcher discovers a flaw
- **Responsible disclosure**: notify vendor first, agree on 90-day window
- Vendor patches, CVE assigned, public disclosure
- **Zero-day**: exploited before vendor knows or before patch exists
- Patch Tuesday (Microsoft), monthly advisory cycles (Red Hat, Ubuntu)

🚨 **Zero-day window:** Between vulnerability discovery/exploitation and patch availability, your only defenses are compensating controls (WAF, network segmentation, logging).

📊 CVSS – Common Vulnerability Scoring System

CVSS v3.1 scores vulnerabilities 0–10 based on exploitability and impact.

Score	Severity	Action
9.0–10.0	**Critical**	Patch immediately (hours–1 day)
7.0–8.9	**High**	Patch within 7 days
4.0–6.9	**Medium**	Patch within 30 days
0.1–3.9	**Low**	Patch in next maintenance cycle
0.0	**None**	Informational

Key CVSS Metrics

```
Attack Vector:     Network (worse) → Local (better)
Attack Complexity: Low (worse) → High (better)
Privileges Req.:   None (worse) → High (better)
```

🔗 NVD & Security Advisories

National Vulnerability Database (NVD)

- `https://nvd.nist.gov/` – searchable database of all CVEs
- Includes CVSS scores, affected versions, references, patches

Distro-Specific Advisories

```
# Ubuntu Security Notices
https://ubuntu.com/security/notices

# Red Hat Security Advisories (RHSA)
https://access.redhat.com/security/security-updates/

# Debian Security Advisories (DSA)
https://www.debian.org/security/

# Subscribe via email or RSS for your distro
```

Checking If Your System Is Affected

```
# Ubuntu: is a specific CVE patched?
ubuntu-security-status
apt-get changelog openssh-server | grep CVE
```

Part 2

🩹 Linux Package Patching

🩹 Patching Workflow: Debian/Ubuntu

```
# 1. Update package index
sudo apt update

# 2. List available upgrades
apt list --upgradable

# 3. List only security updates
sudo apt list --upgradable | grep -i security

# 4. Apply all updates
sudo apt upgrade -y

# 5. Apply dist-upgrade (may add/remove packages)
sudo apt full-upgrade -y

# 6. Apply ONLY security updates
sudo apt-get -s upgrade   # Simulate first
sudo unattended-upgrade --dry-run -d   # Via unattended-upgrades

# 7. Clean up unused packages
sudo apt autoremove --purge -y
sudo apt clean

```

🩹 Patching Workflow: RHEL/CentOS/Fedora

```
# 1. Check for updates
sudo dnf check-update

# 2. List only security updates
sudo dnf updateinfo list security

# 3. List updates for a specific CVE
sudo dnf updateinfo list --cve CVE-2023-38408

# 4. Apply all updates
sudo dnf update -y

# 5. Apply ONLY security updates
sudo dnf update --security -y

# 6. Apply updates for a specific CVE
sudo dnf update --cve CVE-2023-38408

# 7. Check if reboot is required
sudo needs-restarting -r     # Kernel/init only
sudo needs-restarting        # All services needing restart
```

💡 `dnf updateinfo` is powerful: it links RHSA advisory IDs, CVE IDs, and bugzilla numbers directly to packages.

⚙️ Automated Security Updates

Debian/Ubuntu: unattended-upgrades

```
sudo apt install unattended-upgrades

# Configure
sudo dpkg-reconfigure \
  unattended-upgrades

# /etc/apt/apt.conf.d/50unattended-upgrades
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
};
Unattended-Upgrade::AutoFixInterruptedDpkg "true";
Unattended-Upgrade::Remove-Unused-Dependencies "true"
Unattended-Upgrade::Automatic-Reboot "false";  # Care
```

RHEL/CentOS: dnf-automatic

```
sudo dnf install dnf-automatic

# /etc/dnf/automatic.conf
[commands]
upgrade_type = security   # Only security
apply_updates = yes

# Enable timer
sudo systemctl enable --now \
  dnf-automatic.timer

# Check status
systemctl list-timers \
  dnf-automatic.timer
```

⚠️ **Auto-reboot caution:** Enable `Automatic-Reboot` only after setting a maintenance window. Unscheduled reboots during business hours cause outages.

🔄 Kernel Updates & Reboots

Kernel Updates Require a Reboot

```
# Check running kernel
uname -r

# Check installed kernels (Debian/Ubuntu)
dpkg --list | grep linux-image

# Check installed kernels (RHEL)
rpm -qa kernel

# Which kernel will boot next?
sudo grubby --default-kernel      # RHEL
grep submenu /boot/grub/grub.cfg  # Ubuntu

# Check if running kernel matches latest installed
uname -r
ls /boot/vmlinuz-* | tail -1
```

Live Kernel Patching (Enterprise Only)

```
# RHEL: kpatch (applies critical CVE patches without reboot)
sudo dnf install kpatch-patch
sudo kpatch list
```

Part 3

🔍 Vulnerability Scanning Tools

🔍 Lynis – Local Security Auditor

What Lynis Does

- Runs a comprehensive hardening audit locally
- 300+ tests: packages, filesystem, kernel, services, users
- No network needed – scans the host it runs on
- Produces a hardening index score (0–100)
- References CIS Benchmark and STIG checks

Running Lynis

```
# Install
sudo apt install lynis
# or
sudo dnf install lynis

# Full system audit
sudo lynis audit system

# Audit specific category
sudo lynis audit system \
  --tests-from-group authentication

# Non-interactive (CI/scripts)
sudo lynis audit system \
  --quiet --no-colors \
  --logfile /tmp/lynis.log
```

💡 Lynis outputs **Suggestions** and **Warnings**. Warnings are higher priority. Each finding references the test ID (e.g., `AUTH-9308`) – searchable in Lynis documentation.

🔍 Reading Lynis Output

```
[+] Boot and services
————————————————————————————————
  - Service Manager                          [ systemd ]
  - Checking UEFI boot                       [ ENABLED ]
  - Checking presence of GRUB                [ FOUND ]
  - Check if boot loader password is set     [ WARNING ]  ← Needs attention

[+] Authentication
————————————————————————————————
  - Checking password complexity enforcement [ SUGGESTION ]
  - Checking SSH PasswordAuthentication      [ ENABLED ]  ← WARNING
  - Checking accounts without password       [ OK ]

Lynis security scan details:
  Hardening index : 62 [############        ]
  Suggestions     : 28
  Security warnings: 5
```

💡 A hardening index of 60–80 is typical for a newly configured server. Aim for 80+ by addressing warnings and high-priority suggestions. Track the score over time.

🔍 Trivy – Beyond Containers

Trivy scans much more than just Docker images:

```
# Scan a container image (from last week)
trivy image nginx:1.25

# Scan a local filesystem directory
trivy fs /opt/myapp

# Scan a Git repository
trivy repo https://github.com/myorg/myapp

# Scan a running container's rootfs
trivy rootfs /var/lib/docker/overlay2/LAYERID/merged

# Scan Kubernetes cluster
trivy k8s --report summary cluster

# Scan Dockerfile/IaC for misconfigurations
trivy config ./infrastructure/

# Filter by severity
trivy image --severity CRITICAL,HIGH myapp:v1.0

```

🔍 OpenVAS / Greenbone — Network Vulnerability Scanner

What OpenVAS Does

- Remote network-based scanner
- 80,000+ Network Vulnerability Tests (NVTs)
- Scans open ports, services, versions
- Checks for known CVEs, misconfigurations
- Produces detailed HTML/XML reports
- Free (OpenVAS) and commercial (Greenbone Enterprise)

Quick Start

```
# Install via Docker (easiest)
docker run -d \
  -p 9392:9392 \
  --name greenbone \
  greenbone/community-edition

# Wait for feed sync (~20 min first time)
docker logs -f greenbone

# Access web UI
# https://localhost:9392
# Default: admin / admin

# CLI scan
gvm-cli socket \
  --gvm-socket /run/gvmd/gvmd.sock \
  --xml "<get_tasks/>"
```

🔍 `debsecan` & RPM Security Checks

Debian/Ubuntu: debsecan

```
# Install
sudo apt install debsecan

# List CVEs affecting installed packages
debsecan

# Show only fixed (patched) CVEs
debsecan --suite $(lsb_release -cs) --format detail

# Show unfixed vulnerabilities only
debsecan --suite $(lsb_release -cs) --only-fixed=no

# Check if a specific CVE affects the system
debsecan | grep CVE-2023-38408
```

RHEL/CentOS: Security Advisories

```
# List all security advisories
sudo dnf updateinfo list security

# Show details for a specific advisory
sudo dnf updateinfo info RHSA-2023:1234
```

📊 Interpreting Scanner Results

Not All Findings Are Equal — Prioritization Framework

```
Priority 1 (Fix Now):
  CVSS ≥ 9.0 AND Network Attack Vector AND No Auth Required
  → RCE vulnerabilities in internet-facing services

Priority 2 (Fix This Week):
  CVSS 7.0–8.9 OR Known Exploit Available (CISA KEV list)
  → Privilege escalation, sensitive data exposure

Priority 3 (Fix This Month):
  CVSS 4.0–6.9 AND No known exploit
  → Moderate impact, limited exploitability

Priority 4 (Fix in Maintenance Window):
  CVSS < 4.0 OR Requires Physical Access
  → Low risk, compensating controls sufficient
```

CISA Known Exploited Vulnerabilities (KEV)

```
# CISA maintains a list of CVEs with confirmed exploitation in the wild
# https://www.cisa.gov/known-exploited-vulnerabilities-catalog
# Anything on this list: treat as Priority 1 regardless of CVSS
```

Part 4

🛠️ Patch Management at Scale

🛠️ Manual vs Automated Patching

Challenges at Scale

- 10+ servers: manual patching is error-prone
- Different distros in the fleet
- Maintenance windows and change management
- Need to verify patch was applied
- Rollback plan for failed patches
- Audit trail (who patched what, when)

Solutions

- **Ansible** – agentless, SSH-based, idempotent
- **Puppet / Chef / Salt** – agent-based configuration management
- **AWS Systems Manager Patch Manager** – cloud-native
- **Spacewalk / Katello** – on-premises RHEL patch server
- **Landscape** – Ubuntu fleet management

💡 **The golden rule:** Apply patches in a staging environment first, verify functionality, then roll out to production. Never patch production directly.

🤖 Ansible for Fleet Patching

```yaml
# patch-servers.yml
---
- name: Apply security patches
  hosts: all
  become: yes
  serial: 2         # Update 2 servers at a time (rolling update)

  tasks:
    - name: Update package cache (Debian/Ubuntu)
      apt:
        update_cache: yes
      when: ansible_os_family == "Debian"

    - name: Apply security updates (Debian/Ubuntu)
      apt:
        upgrade: yes
        only_upgrade: yes
      when: ansible_os_family == "Debian"

    - name: Apply security updates (RHEL)
      dnf:
        name: "*"
        state: latest
        security: yes
```

🔄 Rollback Strategies

Package-Level Rollback

```
# Debian/Ubuntu: downgrade a package
sudo apt install nginx=1.24.0-1
sudo apt-mark hold nginx       # Prevent future upgrades

# RHEL: install a specific version
sudo dnf install nginx-1.24.0
sudo dnf versionlock add nginx # Lock the version

# RHEL: downgrade
sudo dnf downgrade nginx
```

Snapshot-Based Rollback (Infrastructure Level)

```
# Snapshot the VM before patching (cloud or local)
# AWS: create AMI before update
# Azure: create VM snapshot
# VMware: take snapshot before maintenance window

# Linux: LVM snapshot before major updates
sudo lvcreate -L 5G -s -n root_before_patch /dev/vg0/root

```

📋 CIS Benchmark Compliance Checks

The CIS (Center for Internet Security) Benchmark provides scored hardening guidance for Linux. Lynis and OpenSCAP can check compliance automatically.

```
# Install OpenSCAP
sudo apt install scap-security-guide openscap-scanner

# List available profiles
oscap info /usr/share/xml/scap/ssg/content/ssg-ubuntu2204-ds.xml

# Run CIS Level 1 audit (no changes, just report)
sudo oscap xccdf eval \
  --profile xccdf_org.ssgproject.content_profile_cis_level1_server \
  --report /tmp/cis-report.html \
  /usr/share/xml/scap/ssg/content/ssg-ubuntu2204-ds.xml

# Apply remediation (auto-fix — test in staging first!)
sudo oscap xccdf eval \
  --profile xccdf_org.ssgproject.content_profile_cis_level1_server \
  --remediate \
  /usr/share/xml/scap/ssg/content/ssg-ubuntu2204-ds.xml
```

📊 Patch Management Workflow (End-to-End)

```
1. IDENTIFY
   ├── Subscribe to distro security advisories (email/RSS)
   ├── Run Lynis weekly → new warnings indicate new risks
   ├── Run Trivy on containers before and after image updates
   └── Run debsecan / dnf updateinfo daily (automated)

2. PRIORITIZE
   ├── CVSS ≥ 9.0 → fix within 24 hours
   ├── In CISA KEV list → treat as Critical
   ├── CVSS 7–8.9 → fix within 7 days
   └── CVSS 4–6.9 → fix within 30 days

3. TEST
   ├── Apply patches to staging environment
   ├── Run automated test suite / smoke tests
   └── Verify application functionality

4. DEPLOY
   ├── Snapshot/backup production first
   ├── Apply in rolling fashion (not all at once)
   ├── Reboot if kernel updated
   └── Verify services are healthy after patching

5. VERIFY
```

🔍 Verifying a Patch Was Applied

```
# Check installed version of a package
dpkg -l openssh-server        # Debian
rpm -q openssh-server          # RHEL

# Compare against the advisory's fixed version
# Ubuntu USN says: openssh fixed in 1:9.2p1-2ubuntu3.2
# If your version ≥ that, you're patched

# Verify CVE is no longer reported
debsecan | grep CVE-2023-38408    # Should show nothing if patched

# RHEL: check if advisory was applied
sudo dnf updateinfo info RHSA-2023:1234
rpm -q --changelog openssh | grep CVE-2023-38408

# Check when a package was updated (audit trail)
zcat /var/log/dpkg.log.gz | grep openssh  # Debian
sudo dnf history                           # RHEL — full transaction history
```

✅ **Always verify after patching.** Re-running the scanner after remediation confirms the vulnerability is gone and provides documentation for compliance audits.

📊 Vulnerability Management Metrics

Track these metrics to demonstrate improvement over time:

Metric	Description	Target
MTTD	Mean Time to Detect – how long before you know about a vuln	< 24 hours
MTTR	Mean Time to Remediate – from detection to patching	< 7 days (Critical)
Patch Coverage	% of systems with all Critical/High patches applied	> 95%
Lynis Score	Hardening index	> 80
Open Criticals	CVEs with CVSS ≥ 9.0 with no fix applied	0

💡 These metrics are required for compliance frameworks (PCI-DSS, ISO 27001, SOC 2). Start tracking them early so you have historical data for auditors.

✅ Vulnerability Management Checklist

Detection

- ☐ Subscribe to distro security advisories
- ☐ `unattended-upgrades` or `dnf-automatic` configured
- ☐ `debsecan` / `dnf updateinfo` scheduled daily
- ☐ Lynis run weekly, score tracked
- ☐ Trivy scanning all container images pre-deployment

Remediation

- ☐ Patching SLA defined (Critical: 24h, High: 7d)
- ☐ Staging environment for pre-production testing
- ☐ Snapshots taken before production patches
- ☐ Reboots scheduled in maintenance windows
- ☐ Patch verification after deployment
- ☐ dnf history / dpkg log retained for audit trail

📊 Summary: Vulnerability Scanning & Patching

Key Concepts

1. **CVE**: unique ID for a vulnerability, scored by CVSS
2. **CVSS**: 0–10 score; ≥9 = Critical, ≥7 = High
3. **Lynis**: local hardening auditor with 300+ checks
4. **Trivy**: scans images, filesystems, repos, IaC
5. **OpenVAS**: network-based scanner, 80k+ NVTs
6. **debsecan / dnf updateinfo**: link CVEs to packages
7. **Ansible**: fleet patching at scale, idempotent

Workflow Summary

```
Detect   →  Subscribe to advisories
            Run scanners (Lynis, Trivy)

Prioritize → CVSS score + exposure
                CISA KEV list

Test     →  Staging environment first

Deploy   →  Snapshot → patch → verify
            Rolling updates for HA

Verify   →  Rescan → confirm CVE gone
            Log the patch for compliance
```

🎯 Learning Objectives: Did We Achieve?

By now, you should be able to:

- ✅ Explain what a CVE is and how CVSS scores work
- ✅ Find distro-specific security advisories for Ubuntu and RHEL
- ✅ Apply security updates with `apt` / `dnf` (including CVE-specific patching)
- ✅ Configure automated security updates (`unattended-upgrades` / `dnf-automatic`)
- ✅ Run Lynis and interpret its hardening index and warnings
- ✅ Use Trivy to scan containers, filesystems, and repositories
- ✅ Understand what OpenVAS scans and its legal/ethical constraints
- ✅ Write an Ansible playbook to patch a fleet of servers
- ✅ Describe rollback strategies at the package and infrastructure levels
- ✅ Verify that a CVE has been remediated after patching

🧪 Lab Practice: Scanning & Patching

Exercise 1: System Audit with Lynis

Run `sudo lynis audit system` on your VM. Record the hardening index. Identify the top 3 warnings. Fix one of them (e.g., disable SSH password auth or set a login banner). Re-run Lynis and confirm the warning is gone.

Exercise 2: Package Vulnerability Check

Run `debsecan` (Ubuntu) or `sudo dnf updateinfo list security` (RHEL). Identify the highest-CVSS unfixed CVE. Look it up on NVD. Apply the relevant package update and verify the CVE is resolved.

Exercise 3: Container Scanning with Trivy

Pull `python:3.9` and scan it for CRITICAL CVEs. Then find the closest image with fewer CVEs (try `python:3.11-slim` or `python:3.11-alpine`). Document your findings in a short report.

Exercise 4: Ansible Patch Playbook

Write an Ansible playbook that: updates the package cache, applies security updates, and reboots if required. Run it against your local VM using `ansible-playbook -i localhost, patch.yml --connection=local`.

🔗 Additional Resources

Tools & Databases

- NVD – National Vulnerability Database – CVE search and CVSS scores
- CISA KEV Catalog – Active exploit list
- Lynis – Documentation and test catalog
- Trivy – All scan types and CI integration
- Greenbone Community Edition – OpenVAS setup guide

Standards & Frameworks

- CIS Benchmarks – Free hardening guides (requires registration)
- NIST SP 800-40r4 – Guide to Enterprise Patch Management
- OpenSCAP – Automated compliance scanning

Practice

- Set up a Greenbone Community Edition container and scan your VM
- Integrate Trivy into a GitHub Actions workflow to block vulnerable images
- Work through the CIS Benchmark for your distro section by section

Questions?

End of Semester – Keep Scanning, Keep Patching!